Homework 5: Hypothesis Testing and Monte Carlo

BEE 4850/5850, Spring 2025

Name: Leopold Dorilas

ID: 5362019

Due Date

Friday, 4/17/26, 9:00pm

Overview

Instructions

The goal of this homework assignment is to practicing approaches to extreme value analysis and using information criteria to assess evidence for models.

- Problem 1 asks you to conduct a null hypothesis test using model simulations.
- Problem 1 asks you to use p-values to compare two data-generating hypotheses.
- Problem 3 asks you to use Monte Carlo optimization to identify bidding strategies for The Showcase from The Price is Right.
- Problem 4 asks you to use Monte Carlo optimization to identify house elevation strategies to mitigate flooding damages.

Load Environment

The following code loads the environment and makes sure all needed packages are installed. This should be at the start of most Julia scripts.

```
import pandas as pd
import numpy as np
import random
import matplotlib.pyplot as plt
import scipy
```

Problems

Scoring

- Problem 1 is worth 3 points;
- Problem 2 is worth 5 points;
- Problem 3 is worth 7 points;
- Problem 4 is worth 10 points.

Problem 1 (3 points)

Revisit Problem 1 from Homework 1.

> The underlying question we would like to address is: what is the influence of drinking beer on the likelihood of being bitten by mosquitoes? There is a mechanistic reason why this might occur: mosquitoes are attracted by changes in body temperature and released CO2, and it might be that drinking beer induces these changes. We'll analyze this question using (synthetic) data which separates an experimental population into two groups, one which drank beer and the other which drank only water.

Now we would like to **test** the null hypothesis that drinking beer makes no difference in attracting mosquitoes. Repeat the analysis from HW1. What is the p-value of the observed data under this null hypothesis? If you are interested in testing statistical significance of the proposed alternative hypothesis that beer attracts mosquitoes at the 95% confidence level, what could you conclude that this effect? In other words:

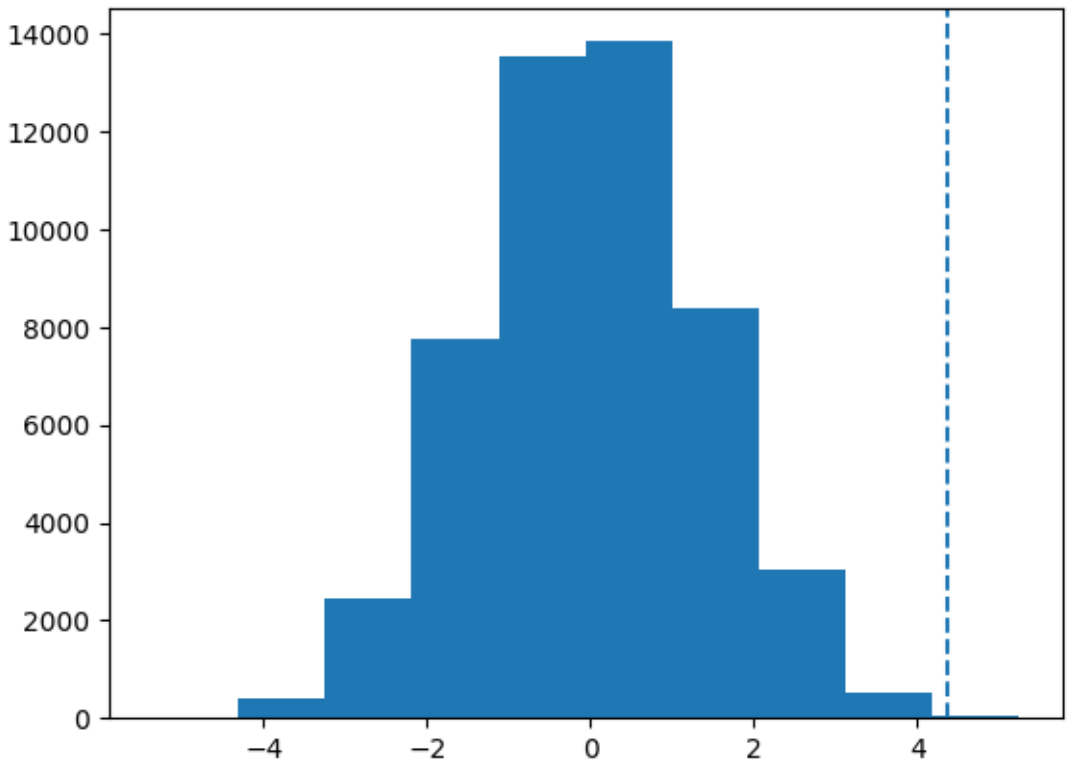
- is the alternative hypothesis statistically significant at this confidence level?
- is the observation more consistent with the alternative hypothesis than the null hypothesis?
- what does your finding imply about any conclusions you might draw about this proposed effect?

```
df = pd.read_csv("data/bites.csv")
print(df.info())
```

```
<class 'pandas.DataFrame'>
RangeIndex: 43 entries, 0 to 42
Data columns (total 2 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   group   43 non-null     str
 1   bites   43 non-null     int64
dtypes: int64(1), str(1)
memory usage: 820.0 bytes
None
```

```
def mean_difference(df):
  beer = df["bites"][df["group"] == "beer"]
  water = df["bites"][df["group"] == "water"]
  return beer.mean() - water.mean() # observed mean difference


def shuffle(df):
  bites = list(df["bites"])
  random.shuffle(bites)
  df["bites"] = bites

control = mean_difference(df)
```

```
x = []
for i in range(50000):
  shuffle(df)
  x.append(mean_difference(df))

fig, ax = plt.subplots()
ax.hist(x)
ax.axvline(control, linestyle='dashed')
plt.show()
```




```
p_h = np.mean(np.array(x) >= control)

print(f"Observed mean difference: {control}")
print(f"p-value: {p_h}")
```

```
Observed mean difference: 4.37777777777778
p-value: 0.00058
```

The alternative hypothesis is statistically significant at the 95% confidence level. We can conclude that beer drinking has a significant impact on mosquito bite presence.

Problem 2 (5 points)

Revisit Problem 5 from Homework 1.

> You are trying to detect how prevalent cheating was on an exam. You are skeptical of the efficacy of just asking the students if they cheated. You are also concerned

about privacy — your goal is not to punish individual students, but to see if there are systemic problems that need to be addressed. Someone proposes the following interview procedure, which the class agrees to participate in:

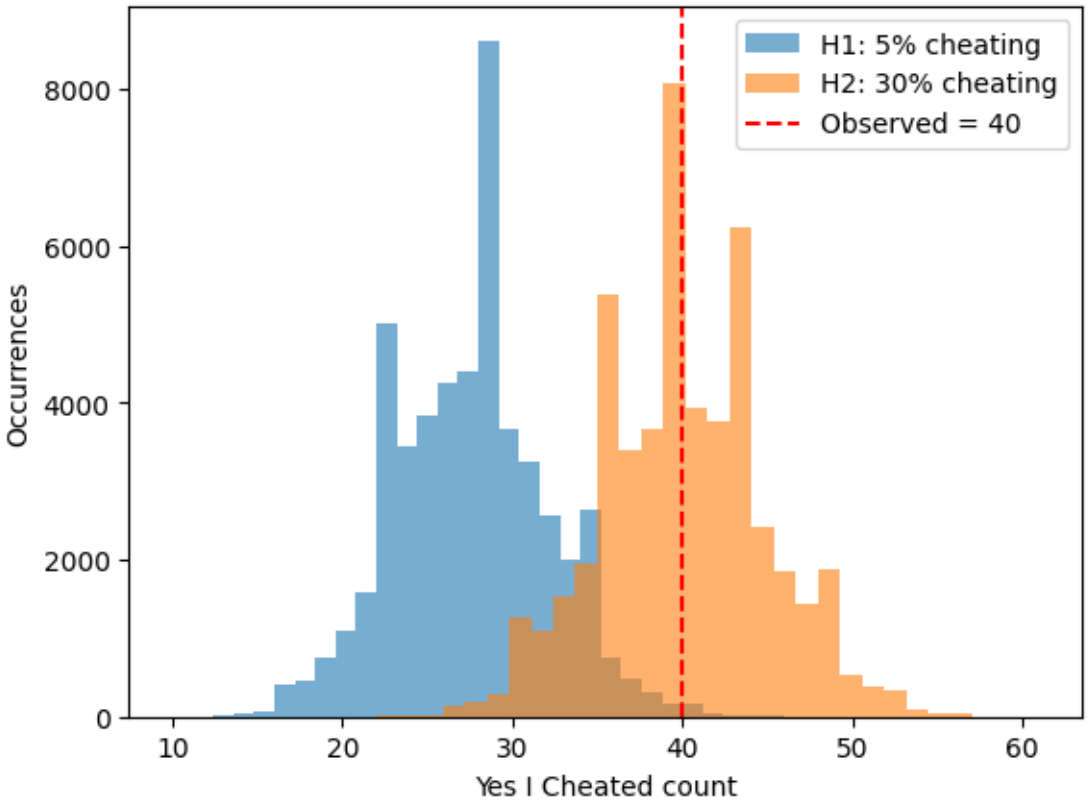
Each student flips a fair coin, with the results hidden from the interviewer. The student answers honestly if the coin comes up heads. Otherwise, if the coin comes up tails, the student flips the coin again, and answers "I did cheat" if heads, and "I did not cheat", if tails.

You have a hypothesis that cheating was not prevalent, and the proportion of cheaters was no more than 5% of the class; in other words, we expect 5 "true" cheaters out of a class of 100 students. Our TA is more jaded and thinks that cheating was more rampant, and that 30% of the class cheated. The proposed interview procedure is noisy: the interviewer does not know if an admission means that the student cheated, or the result of a heads. However, it gives us a data-generating process that we can model and analyze for consistency with our hypothesis and that of the TA.

Repeat this analysis and suppose that your procedure turned up 40 "yes" answers. What is the p-value of having seen this data under both hypotheses? If you wanted to draw a conclusion at a 95% confidence level, what could you conclude about the level of cheating in the class?

```
def simulate(p):
    cheaters = 0
    for i in range(100):
        cheating = random.random() < p
        r = random.random()
        if r > 0.5:
            if cheating:
                cheaters += 1
        else:
            r = random.random()
            cheaters += r > 0.5
    return cheaters

p5  = [simulate(0.05) for i in range(50_000)]
p30 = [simulate(0.30) for i in range(50_000)]

observed_yes = 40

plt.figure()
plt.hist(p5,  bins=30, alpha=0.6, label="H1: 5% cheating")
plt.hist(p30, bins=30, alpha=0.6, label="H2: 30% cheating")
```

```
plt.axvline(observed_yes, color="red", linestyle="--", label=f"Observed = {observed_yes}")
plt.xlabel("Yes I Cheated count")
plt.ylabel("Occurrences")
plt.legend()
plt.show()

p_val_h1 = sum(v >= observed_yes for v in p5)  / len(p5)
p_val_h2 = sum(v >= observed_yes for v in p30) / len(p30)
```




```
print(f"Hypothesis 1 p-value:{p_val_h1}")
print(f"Hypothesis 2 p-value: {p_val_h2}")
```

```
Hypothesis 1 p-value:0.0043
Hypothesis 2 p-value: 0.536
```

We got a p-value way above 0.05 for the second hypothesis (30% cheating) and a p-value below 0.05 for the first hypothesis (5% cheating). We can say that it is more likely that 30% of the class cheated.

Problem 3 (7 points)

The Showcase is the final round of every episode of The Price is Right, matching the two big winners from the episode. Each contestant is shown a "showcase" of prizes, which are usually some combination of a trip, a motor vehicle, some furniture, and maybe some other stuff. They then each have to make a bid on the retail price of the showcase. The rules are:

- an overbid is an automatic loss;
- the contest who gets closest to the retail price wins their showcase;
- if a contestant gets within $250 of the retail price and is closer than their opponent, they win both showcases.

You have been selected to appear on the show. As preparation, you have documented historical prizes and their values. Once you're shown the showcase, you realize that the distribution of values for this spread of prizes has historically followed a truncated normal distribution with mean $31,000 and standard deviation $4,500, truncated between $15,000 and $42,000.

You can make the following assumptions about the probability of winning and payouts:

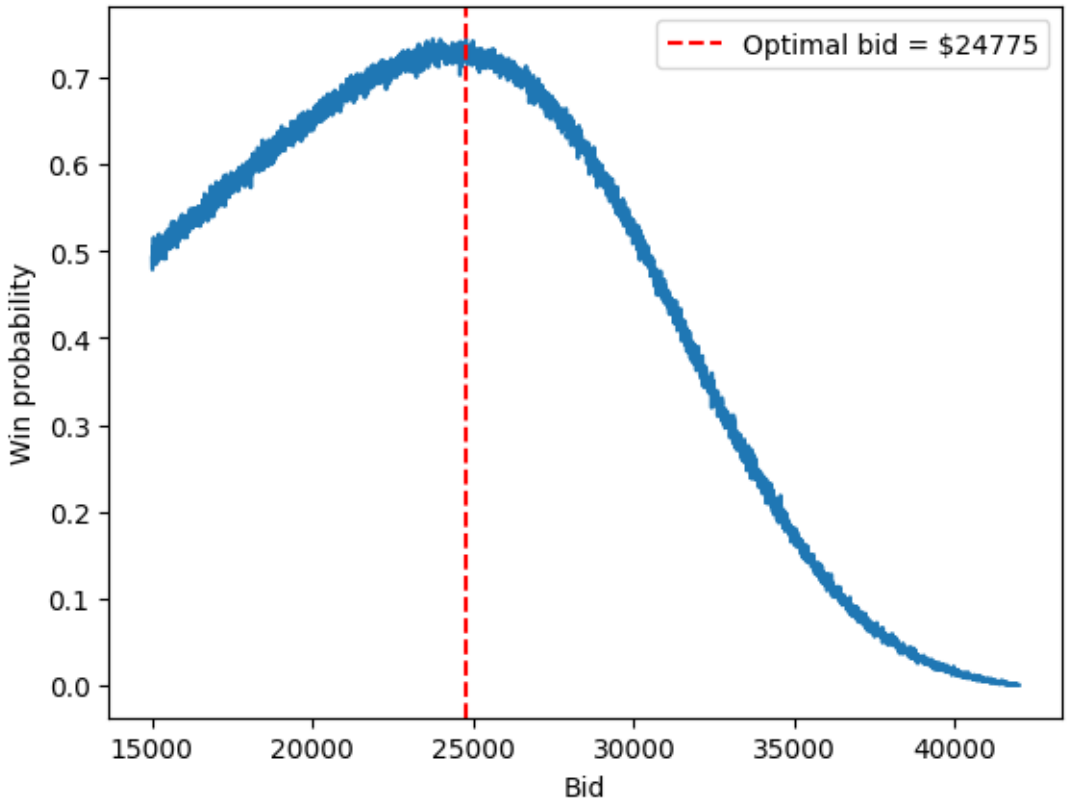
- If you bet the exact amount, you win with probability 1 and win both showcases; the value is the double of the single showcase value.
- If you did not win both showcases but bid under the showcase value, the probability of being outbid increases linearly as the distance between your bid and the value increases (in other words, if you bid the exact value, you win with probability 1, and if you bid $0, you win with probability 0).
- If you overbid, you automatically lose, with value $0.

```
from scipy.stats import truncnorm

mu, sigma = 31000, 4500
a, b = (15000 - mu) / sigma, (42000 - mu) / sigma
showcase_dist = truncnorm(a, b, loc=mu, scale=sigma)
simulations = 5000
def simulate_showcase(bid):
    V = showcase_dist.rvs(simulations)
    overbid = bid > V # mask
    p_win = np.where(overbid, 0.0, bid / V)
    won = np.random.random(simulations) < p_win
    win_both = won & (np.abs(V - bid) <= 250)
    payoff = np.where(win_both, 2 * V, np.where(won, V, 0.0))
    return won.mean(), payoff.mean()
```

Problem 3.1

Using Monte Carlo simulation, find the wager that maximizes your probability of winning. You will need to write a function which computes the winning probability for a given bet, then optimize that function. Make sure to justify the sample size that you use in your Monte Carlo analysis.

```
bids = np.arange(15000, 42000, 5) # granular increments of 5, still settles around the same p
win_probs = [simulate_showcase(bid)[0] for bid in bids]

best_idx = np.argmax(win_probs)
best_bid = bids[best_idx]

fig, ax = plt.subplots()
ax.plot(bids, win_probs)
ax.axvline(best_bid, color="red", linestyle="--", label=f"Optimal bid = ${best_bid}")
ax.set_xlabel("Bid")
ax.set_ylabel("Win probability")
ax.legend()
plt.show()

print(f"Bid maximizing win probability: ${best_bid}")
print(f"Probability: {win_probs[best_idx] * 100}%")
```



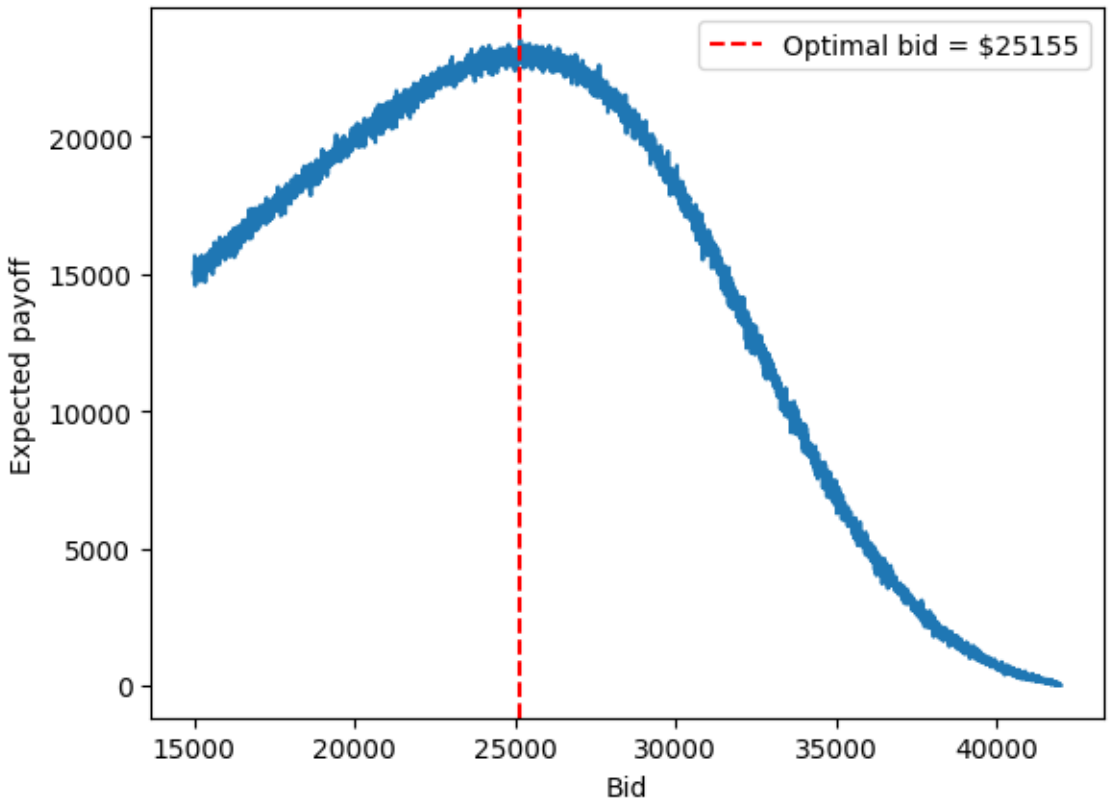

```
Bid maximizing win probability: $24775
Probability: 74.44%
```

Problem 3.2

Using Monte Carlo simulation, find the wager that maximizes your expected winnings. Is this the same as in Problem 3.1? Why do you think it is or is not? Make sure to justify the sample size that you use in your Monte Carlo analysis.

```
expected_payoffs = [simulate_showcase(bid)[1] for bid in bids]

best_payoff_idx = np.argmax(expected_payoffs)
best_payoff_bid = bids[best_payoff_idx]

fig, ax = plt.subplots()
ax.plot(bids, expected_payoffs)
ax.axvline(best_payoff_bid, color="red", linestyle="--", label=f"Optimal bid = ${best_payoff
ax.set_xlabel("Bid")
```

```
ax.set_ylabel("Expected payoff")
ax.legend()
plt.show()

print(f"Bid maximizing expected payoff: ${best_payoff_bid}  (E[payoff]  ${int(expected_payof
```




```
Bid maximizing expected payoff: $25155  (E[payoff]  $23532)
```

The payoff maximizing bid is higher than the win probability maximizing bid by around four hundred dollars. Overbidding is an automatic loss, so going closer to the retail price is riskier, but being within the 250 dollar threshold / boundary makes the riskier, higher bid worth it to maximize the payoff. The bid that maximizes the win probability is more cautious and goes lower to avoid overbidding altogether. This comes with around 5400 simulation samples

Problem 4

You have been asked by a client to assess the risks of flooding to their home (which is valued at $400,000 and only floods when the stream water levels exceed 1.5m). They would like to know whether it would be cost-effective to elevate their home to reduce flood risks. After some hydrologic analysis, you have developed a 40-year record of annual maximum water levels (in m) at the nearby stream.

You also have a depth-damage function for the fraction of the home's value which is damaged at varying flood depths:

$$d(h) = \mathbb{I}[h > 0]\frac{1}{1 + \exp(-k(x - x_0))},$$

where $k = 1.25$ and $x_0 = 2$[1]. The graph of this function is given in the figure below.

[1] As a reminder, the indicator function $\mathbb{I}[h > 0]$ is 0 when the condition is not satisfied (in this case, when $h = 0$) and 1 when it is satisfied.

In this problem, you will need to calculate the **net present value (NPV)** of flooding damages, which converts damages over time to a present value using a discount rate[1] For example, if your discount rate is 4%, a dollar next year is worth the equivalent of about 96 cents today. More generally, with a discount rate of γ (as a decimal), a dollar of benefits in t years is worth $\$(1 - \gamma)^t$ today.

The NPV of a sequence of money x_t with discount rate γ is the sum of all of the discounted values, that is,

$$NPV = \sum_{t=0}^{T} x_t(1 - \gamma)^t,$$

where T is the time horizon. We'll use a discount rate of 4% in this problem, which is typical for this type of problem, and a design horizon of 30 years.

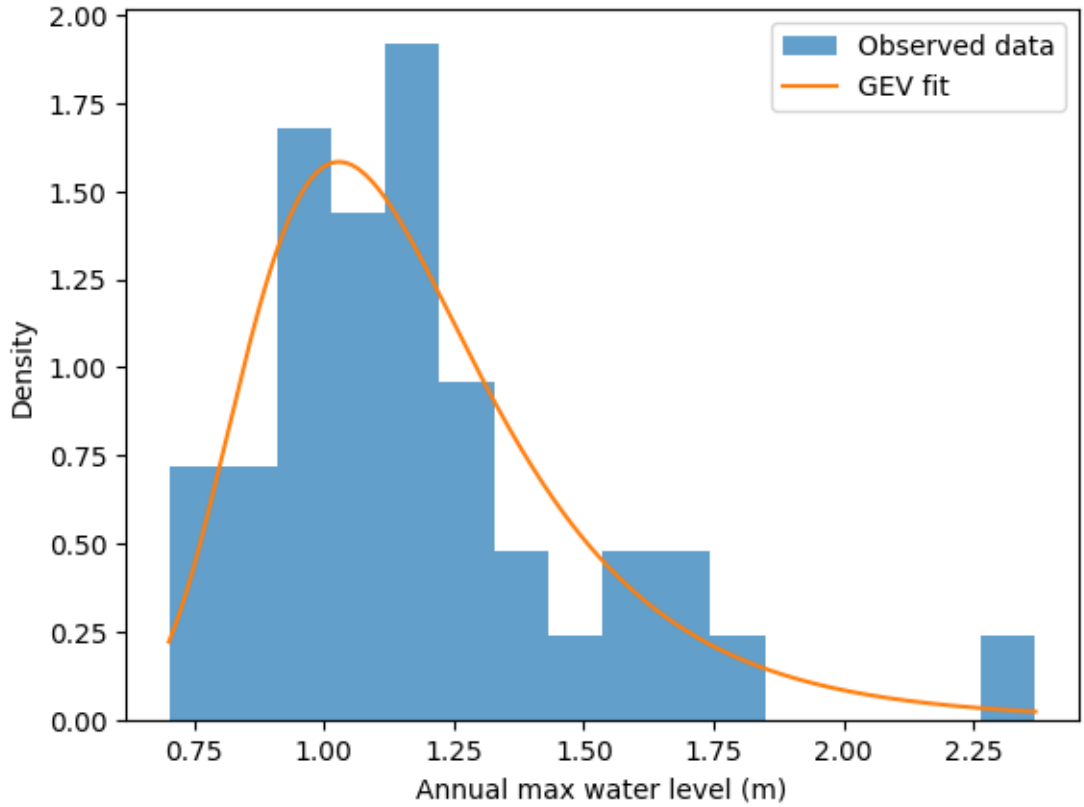
For this problem, we will assume that the cost of elevating the house Δh m is $C(\Delta h) = \mathbb{I}[h > 0](100,000 + 2,000\Delta h)$.

Problem 4.1

Fit a GEV distribution to the data in `water_depths.csv`. Plot a histogram of the data and the fitted distribution. How well does this distribution fit the data?

```python
from scipy.stats import genextreme

depths = pd.read_csv("data/water_depths.csv")["depths"].values

shape, loc, scale = genextreme.fit(depths)
print(f"GEV parameters: shape = {shape}, loc = {loc}, scale = {scale}")

fig, ax = plt.subplots()
ax.hist(depths, bins=16, density=True, alpha=0.7, label="Observed data")
x_range = np.linspace(depths.min(), depths.max(), 300)
ax.plot(x_range, genextreme.pdf(x_range, shape, loc=loc, scale=scale), label="GEV fit")
ax.set_xlabel("Annual max water level (m)")
ax.set_ylabel("Density")
ax.legend()
plt.show()
```

```
GEV parameters: shape = -0.05447796093852186, loc = 1.041486818572033, scale = 0.232735574175
```



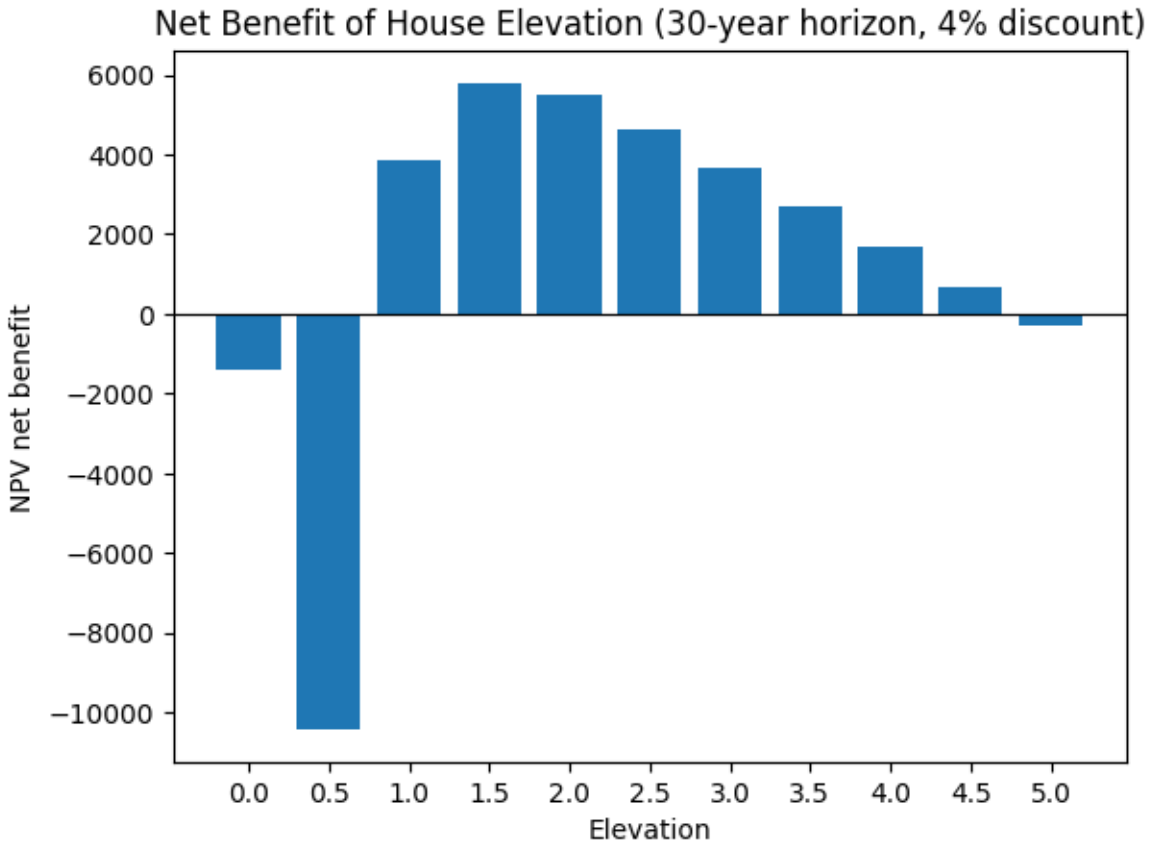
Problem 4.2

Use Monte Carlo simulation to estimate the NPV of the benefits of elevation levels between 0 and 5m (you don't have to consider increments finer than 0.5m). What elevation heights pass a cost-benefit test (*e.g.* the net benefits of elevation (benefits of avoiding flooding minus cost of elevation) are greater than 0)? You can assume that the costs of elevation are all up front (that is, in year 0). What height (including a possible elevation of 0m) maximizes the NPV of net benefits?

[1] Discount rates reduce future monetary values to reflect the time-value of money; that is, you would rather have a bit less money today than none today and more next year, as you could save or invest that money. The actual choice of discount rates is the subject of much economic theory and plays an important role in environmental decision-making, particularly for multi-generational investments such as climate mitigation. For example, with a relatively large discount rate (such as 7%), all costs and benefits after a decade are effectively zero, which means it never appears cost-effective to e.g. reduce fossil fuel emissions.

```
simulations = 10000
T = 30
gamma = 0.04
home_value = 400000
k_dd, x0_dd = 1.25, 2.0
flood_threshold = 1.5
discount = (1 - gamma) ** np.arange(T)

def depth_damage(h):
    return np.where(h > 0, 1 / (1 + np.exp(-k_dd * (h - x0_dd))), 0.0)

def elevation_cost(dh):
    return 0.0 if dh == 0 else 100000 + 2_000 * dh

def npv_damages(elevation):
    water = genextreme.rvs(shape, loc=loc, scale=scale, size=(simulations, T))
    flood_depth = water - flood_threshold - elevation
    annual_damage = home_value * depth_damage(flood_depth)
    return (annual_damage * discount).sum(axis=1).mean()

elevations = np.arange(0, 5.5, 0.5)
baseline_npv = npv_damages(0)

rows = []
for dh in elevations:
    dmg_npv = npv_damages(dh)
    cost = elevation_cost(dh)
```

```
    net_benefit = (baseline_npv - dmg_npv) - cost
    rows.append((dh, dmg_npv, cost, net_benefit))

best = max(rows, key=lambda r: r[3])

fig, ax = plt.subplots()
net_benefits = [r[3] for r in rows]
ax.bar([f"{r[0]}" for r in rows], net_benefits)
ax.axhline(0, color="black", linewidth=0.8)
ax.set_xlabel("Elevation")
ax.set_ylabel("NPV net benefit")
ax.set_title("Net Benefit of House Elevation (30-year horizon, 4% discount)")
plt.show()
```



```
print(f"Baseline NPV damages: ${baseline_npv}")
print(f"Optimal elevation: {best[0]}m  (net benefit  ${best[3]})")
```

```
Baseline NPV damages: $111122.27686643387
Optimal elevation: 1.5m  (net benefit  $7207.396660580431)
```